

Tecnológico de Monterrey

Campus Puebla

M2. Tarea Pirámide

José Mariano Cariño Martínez A01739206

Modelación de sistemas multiagentes con gráficas computacionales

group 501

13/10/2026

Descripción técnica

La solución convierte cinco puntos 3D en líneas sobre la pantalla, 60 veces por segundo. Tres archivos principales permiten la graficación:

main.py: abre la ventana, corre el ciclo principal y llama a update y render.

OpMat.py: guarda una matriz 4x4 donde se acumulan las operaciones necesarias para animar el objeto 3D.

piramide.py: define vértices y aristas, proyecta a 2D con ayuda de una clase externa de cámara y dibuja cada arista con el algoritmo de Bresenham (también implementado en una función aparte).

El sistema de coordenadas es de mano izquierda: X hacia la derecha, Y hacia arriba y Z hacia adentro de la pantalla. La pirámide es unitaria: su vértice 0 está en el origen, su base mide 1 x 1 y su altura es 1.

El flujo que sigue cada vértice en cada cuadro es el siguiente:

Definición: el vértice en coordenadas del modelo, por ejemplo el vértice 0, (0.0, 0.0, 0.0), que coincide con el origen.

Matriz de modelado: se escala, se traslada el pivote al origen, se rota alrededor del eje Y, se regresa el pivote a su posición y, al final, se coloca el objeto en el mundo.

Proyección en perspectiva: x e y se dividen entre la distancia a la cámara.

Conversión a píxeles de pygame, recordando que su origen está arriba a la izquierda.

Bresenham une con píxeles los dos extremos de cada arista.

Coordenadas homogéneas

Se usa una matriz 4x4 para un punto 3D porque así la traslación también se vuelve una multiplicación y puede combinarse en una sola matriz con las demás operaciones (escala y rotación). Para ello el punto se escribe como (x, y, z, 1).

Cada transformación es una matriz 4x4 que se multiplica por el punto, y todas parten de la matriz identidad que deja el método reset(). Estas operaciones se definen en la clase OpMat3D. En la rotación hay que elegir el eje, porque la matriz es distinta para cada uno: la coordenada del eje de giro es la que no cambia (en rotate_y, la Y queda igual). En esta actividad aún no se implementan rotaciones alrededor de un eje arbitrario.

El orden de las llamadas importa, porque la multiplicación de matrices no es conmutativa. Cada método multiplica la nueva matriz por la izquierda (M = R · M), de modo que el orden del código es el orden en que se aplican las operaciones al punto: escalar, rotar y trasladar.

Rotación alrededor de un pivote

Se implementó la variable pivote, que guarda el punto P alrededor del cual gira el objeto. En piramide.py se traslada todo el objeto para que P coincida con el origen, se rota alrededor del eje Y y se regresa P a su posición original, es decir, T(P) · R_y(θ) · T(-P). El eje de giro es la línea vertical que pasa por P, y P permanece fijo. Como la escala se aplica primero, el pivote también se escala. Cuando P es el origen, como en el vértice 0, las dos traslaciones se cancelan.

La cámara

La proyección parte de la idea de que lo lejano se ve más pequeño, por lo que x e y se dividen entre la distancia a la cámara. Esto ocurre en el método projectar:

x_proj = (x · d) / z', y_proj = (y · d) / z', con z' = z + 4

donde z' es la distancia del punto a la cámara y d la distancia focal en píxeles.

Conversión a pantalla

Después de proyectar, las coordenadas se trasladan al sistema de la ventana, cuyo origen está arriba a la izquierda y donde Y crece hacia abajo:

x_pantalla = W/2 + x_proj, y_pantalla = H/2 − y_proj

Se resta y_proj para voltear la altura.

Bresenham

El algoritmo de Bresenham dibuja la línea entre dos píxeles usando solo enteros: avanza un píxel a la vez y mantiene una variable de error que decide si el siguiente paso es en x, en y o en ambos, usando únicamente sumas y comparaciones.

Link a Vídeo Explicativo:

https://youtu.be/xS5oX8f_IbA?si=EYANBopGYjXr-Ew-